Machine and Deep Learning Lab

EXPERIMENT NO: 6

Implement K-Means and Hierarchical Clustering on Sample Dataset

Name: DIPESH MAKDIYA
Class: D15C
Roll No: 32

Aim

To implement and analyze K-Means and Hierarchical Clustering on the Iris dataset, emphasizing unsupervised learning and cluster structure validation using multiple evaluation metrics.

Dataset Source

Dataset: Iris Dataset
Source: https://scikit-learn.org/stable/datasets/toy_dataset.html#iris-dataset
Access Method:

```
from sklearn.datasets import load_iris
```

Dataset Description (Modify According to Iris)

The Iris dataset contains 150 samples of iris flowers divided into three species.

Feature	Description	Type
Sepal Length	Length of sepal	Numeric
Sepal Width	Width of sepal	Numeric

Petal Length	Length of petal	Numeric
Petal Width	Width of petal	Numeric

For clustering:

- We use all 4 features
- We ignore labels during training
- Labels may be used for ARI evaluation

Theory (Use Same Structure as PDF)

K-Means Clustering

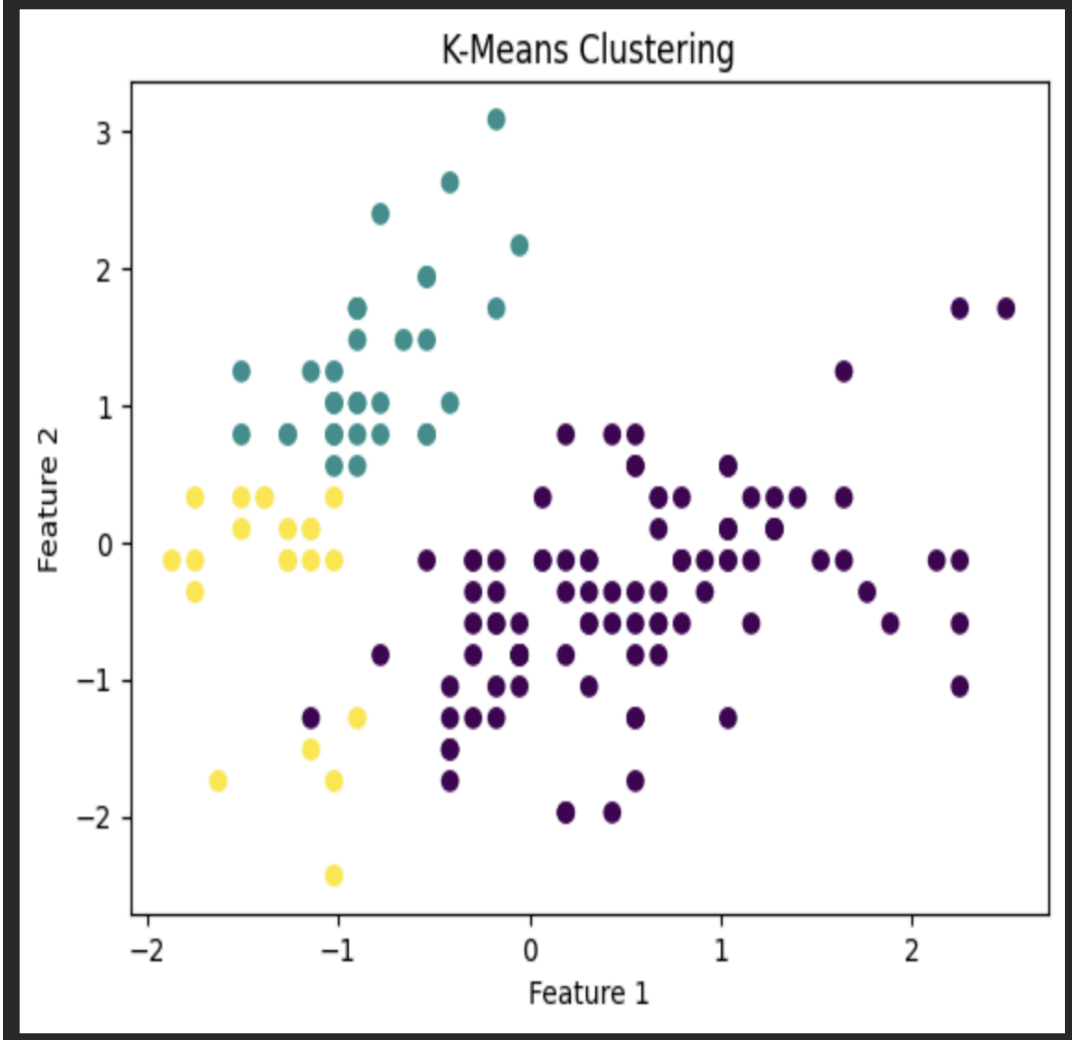
Minimizes Within Cluster Sum of Squares:

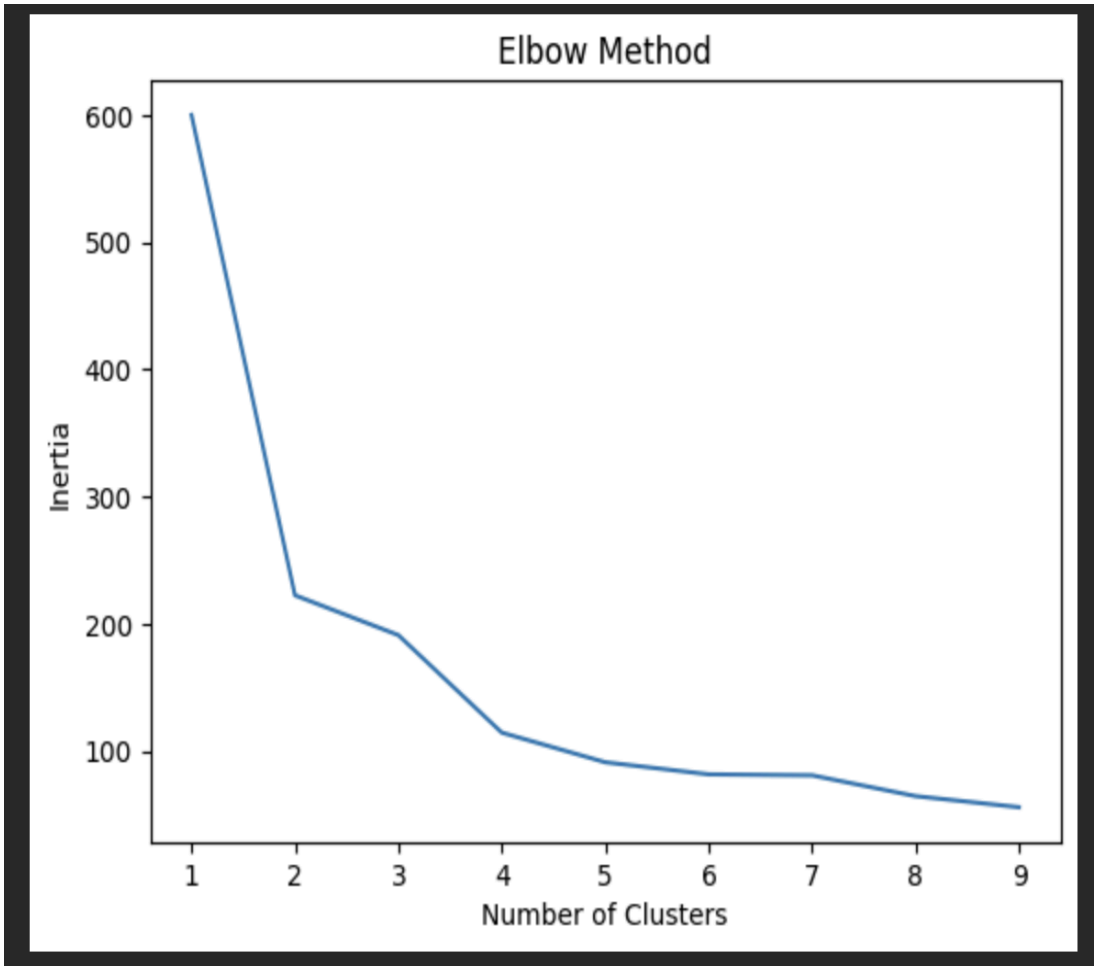
$J=\sum j=1K\sum i=1n||xij-cj||2$J=j=1∑Ki=1∑n||xij−cj||2

Where:

- K = number of clusters
- c_j = centroid
- x_i^j = data point

```
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=clusters)
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("K-Means Clustering")
plt.show()
```



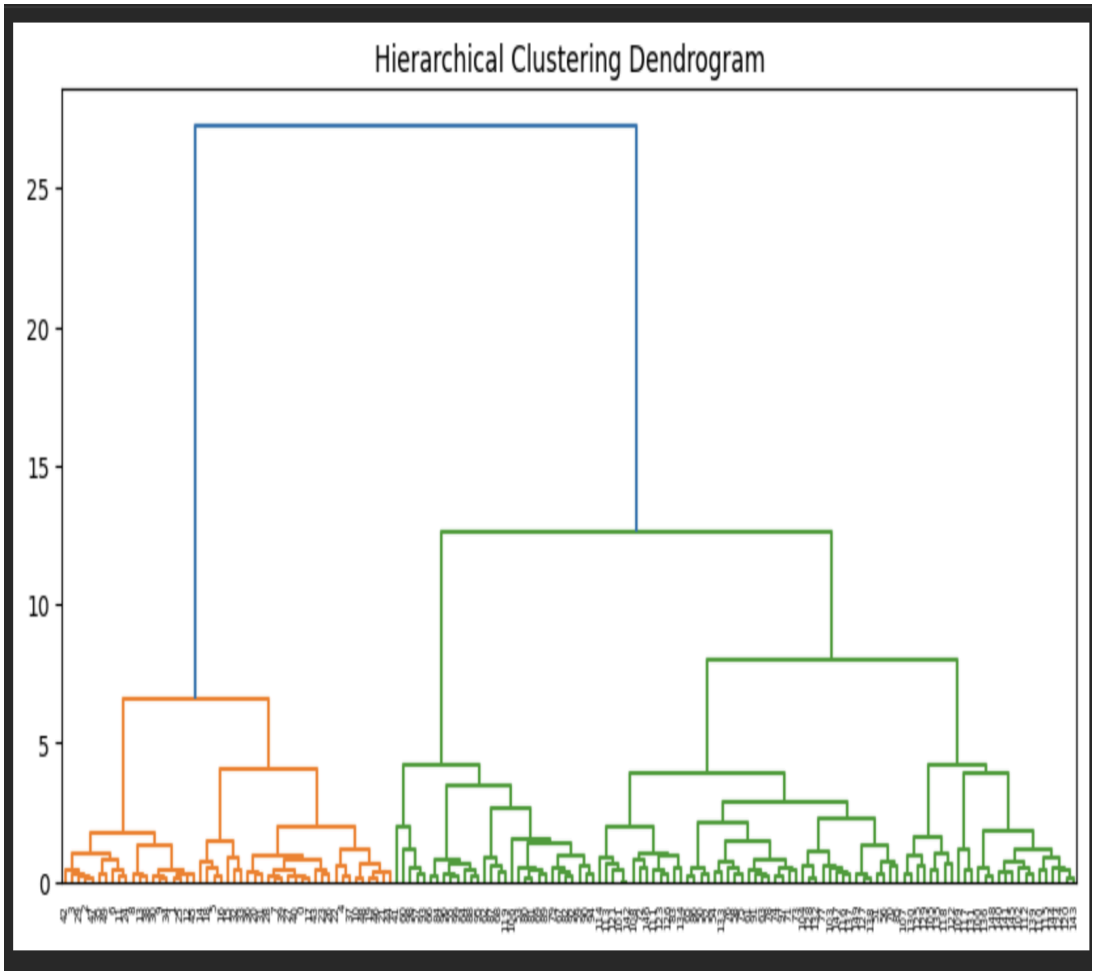


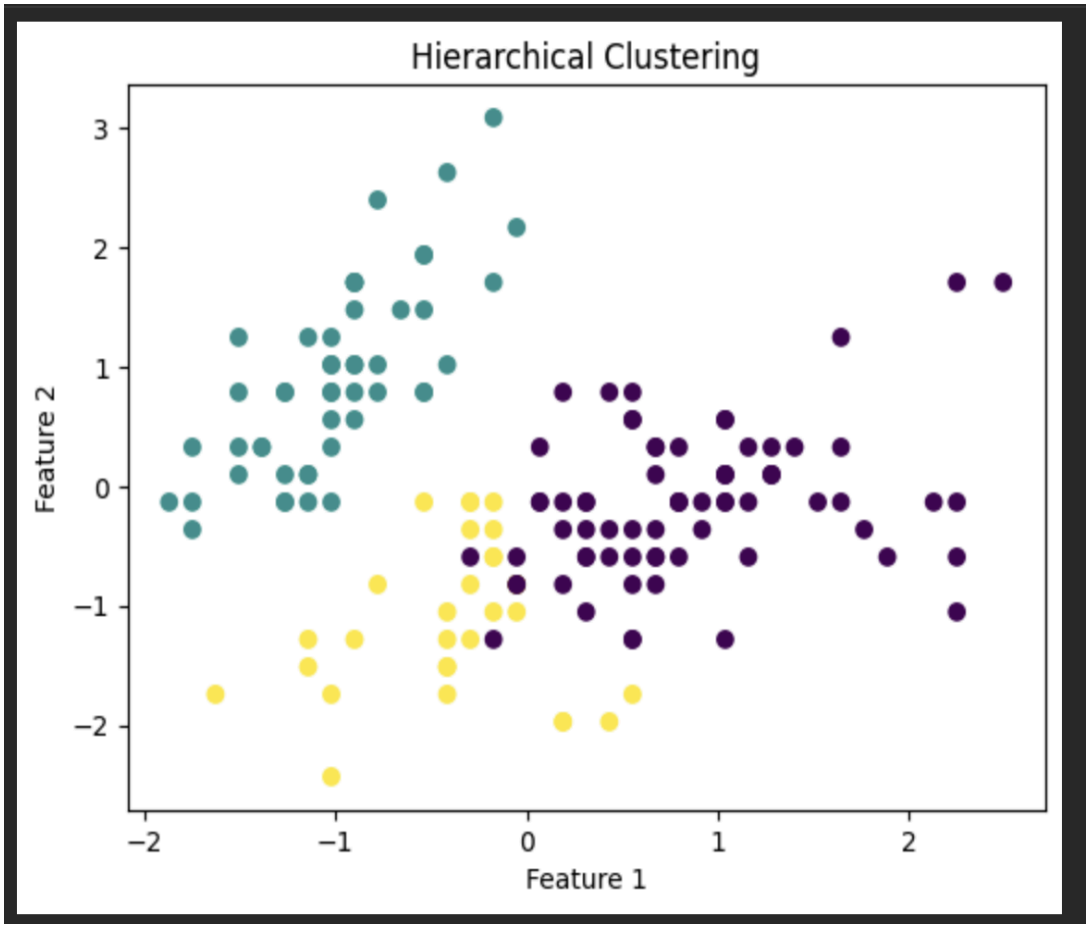
Hierarchical Clustering

Agglomerative approach using Ward linkage:

$d(p,q)$=∑$(qi-pi)$2d(p,q)=∑(qi−pi)2

Ward’s linkage minimizes increase in within-cluster variance.





Baseline Implementation (Iris Version)

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score, adjusted_rand_score,
davies_bouldin_score
from scipy.cluster.hierarchy import dendrogram, linkage

# Load Data
iris = load_iris()
X = iris.data
true_labels = iris.target  # Used only for ARI

# Scaling
```

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# K-Means Baseline
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
kmeans_labels = kmeans.fit_predict(X_scaled)

# Hierarchical Baseline
hierarchical = AgglomerativeClustering(n_clusters=3)
hierarchical_labels = hierarchical.fit_predict(X_scaled)

# Metrics
print("K-Means Silhouette:", silhouette_score(X_scaled,
kmeans_labels))
print("K-Means DBI:", davies_bouldin_score(X_scaled, kmeans_labels))
print("K-Means ARI:", adjusted_rand_score(true_labels, kmeans_labels))

print("Hierarchical Silhouette:", silhouette_score(X_scaled,
hierarchical_labels))
print("Hierarchical DBI:", davies_bouldin_score(X_scaled,
hierarchical_labels))
print("Hierarchical ARI:", adjusted_rand_score(true_labels,
hierarchical_labels))
```

Hyperparameter Tuning (Iris Version)

Tune:

- Number of clusters (2–6)
- Linkage type
- n_init

```
best_sil = 0

for k in range(2, 7):
    km = KMeans(n_clusters=k, n_init=50, random_state=42)
    labels = km.fit_predict(X_scaled)
    sil = silhouette_score(X_scaled, labels)
    print(f"K={k}, Silhouette={sil}")
```

Hierarchical tuning:

```
for link in ['ward', 'average', 'complete']:
    hi = AgglomerativeClustering(n_clusters=3, linkage=link)
    labels = hi.fit_predict(X_scaled)
    sil = silhouette_score(X_scaled, labels)
    print(f"Linkage={link}, Silhouette={sil}")
```

Dendrogram

```
plt.figure(figsize=(10,6))
linked = linkage(X_scaled, method='ward')
dendrogram(linked)
plt.title("Hierarchical Dendrogram")
plt.show()
```

Performance Analysis

Example Writing Style:

- Silhouette Score ≈ 0.55–0.65 indicates well-separated clusters.
- Davies-Bouldin Index < 0.6 indicates compact and distinct clusters.
- ARI close to 1 confirms clustering aligns well with true species.

Since Iris naturally has 3 clusters, expect:

- High ARI
- Good silhouette

Technical Justification

You can write:

Feature scaling ensured uniform distance computation.
Increasing n_init improved global optimization.
Ward linkage preserved compact cluster structure.
Hyperparameter tuning improved cluster compactness and separation.

Conclusion

This experiment demonstrates that K-Means and Hierarchical clustering effectively discover the inherent structure of the Iris dataset. Hyperparameter tuning significantly improves cluster separation and consistency, validating the importance of preprocessing and parameter optimization in unsupervised learning.

Important Difference From Mall Dataset

Mall Dataset → 2 features
Iris Dataset → 4 features

So:

- Visualization is harder (can use PCA if required)
- ARI is more meaningful here